Database Design Analysis

Normalization, redundancy, anomalies, dependencies, and a normalized hotel database

Prepared by: Hemanth Krishna S

Project: Hotel Management System. Output convention: result tables are expected outputs from the fixed rows in the shared SQL script; they are not screenshots of MySQL execution.

1. Aim and scope

This practical examines how a single flat hotel register can duplicate guest, room, hotel, room-type, booking, and payment details. It identifies insertion, update, and deletion anomalies, lists functional dependencies, and maps the data into nine relations in Third Normal Form (3NF).

A booking may have multiple payments or services. Keeping all information in one row can repeat the same booking and guest facts. The final design stores each entity once and uses foreign keys and a junction table for the many-to-many relationship between bookings and services.

2. Functional dependencies

Determinant	Dependent attributes	Reason
hotel_id	hotel_name, city, phone	Identifies one hotel.
room_type_id	type_name, max_occupancy, base_rate	Identifies one room category and its standard rate.
guest_id	guest_no, full_name, email, phone, registered_on	Guest details are independent of individual bookings.
room_id	hotel_id, room_type_id, room_number, floor_no, room_status	Identifies a physical room.
booking_id	guest_id, room_id, check_in, check_out, booking_status	Identifies one reservation/stay record.
service_id	service_name, unit_price	Identifies a hotel service and its unit price.
payment_id	booking_id, payment_date, amount, payment_method, payment_status	Identifies one payment transaction.
(booking_id, service_id)	quantity	The composite key identifies a service attached to a booking.

The flat register's composite key could be (booking_id, service_id) when each row represents one booking-service pair. Guest, room, hotel, and booking facts would then be repeated for every service. Partial and transitive dependencies make the flat relation unsuitable.

Normalization steps

Stage	Design change
1NF	Store atomic values; list each service as a separate booking_services row rather than a comma-separated list.
2NF	Move guest and booking facts out of a relation keyed by both booking_id and service_id.
3NF	Store hotel, room type, guest, room, service, payment, and booking facts in separate relations; use keys to connect them.

3. Anomaly demonstration

The following simplified table demonstrates duplicate booking facts. Run it in MySQL Workbench to see how the same guest and room details repeat across service rows.

```
CREATE TEMPORARY TABLE hotel_register_raw (
  booking_id INT NOT NULL,
  service_id INT NOT NULL,
  guest_name VARCHAR(100) NOT NULL,
  room_number VARCHAR(10) NOT NULL,
  service_name VARCHAR(80) NOT NULL,
  PRIMARY KEY (booking_id, service_id)
);
INSERT INTO hotel_register_raw VALUES
(1001,1,'Ananya Rao','101','Breakfast'),
(1001,3,'Ananya Rao','101','Laundry'),
(1002,1,'Rahul Nair','201','Breakfast');
SELECT booking_id, guest_name, room_number, COUNT(*) AS service_rows
FROM hotel_register_raw
GROUP BY booking_id, guest_name, room_number;
UPDATE hotel_register_raw SET room_number='101A'
WHERE booking_id=1001 AND service_id=1;
SELECT DISTINCT room_number FROM hotel_register_raw WHERE booking_id=1001;
DROP TEMPORARY TABLE hotel_register_raw;
```

Update anomaly: changing only one repeated row leaves conflicting room values. Insertion anomaly: a guest cannot be stored naturally if the flat table requires a booking and service. Deletion anomaly: deleting the only row for a service could also remove the only stored occurrence of its booking facts.

4. Final normalized relations

The project uses nine tables: hotels, room_types, guests, staff, rooms, bookings, services, booking_services, and payments. Primary keys identify records; foreign keys preserve relationships. This structure reduces duplication and supports multi-table reports.

5. Conclusion

Normalization improves consistency by keeping each fact in the relation where it belongs. The nine-table design is suitable for a small hotel system that tracks rooms, guests, bookings, services, staff, and payments.